UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE (UFRN)

DEPARTAMENTO DE COMPUTAÇÃO E AUTOMAÇÃO (DCA)

DCA3604 - BANCO DE DADOS

Sistema Integrado de Gestão e Triagem Animal

Componentes do Grupo:

Ester Bendicto Wanek Teixeira
Felipe Augusto Figueiredo da Silva
Gabriel Figueiredo Gonzaga
Iago Rafael Ramos Cantuaria Moreira

Docente: Prof. Matheus Vitor de Andrade Pedrosa

Natal / RN
2026

Sumário

1 Apresentação do Projeto **2**

2 Descrição dos Dados Reais Utilizados e sua Origem **2**

2.1 Especificações do Dataset 2

2.2 Processo de Ingestão e Carga (ETL) 3

3 Análise dos Dados Reais: Contexto e Descobertas **3**

3.1 Pergunta 1: Qual o volume e a variação diária de acolhimentos? 3

3.2 Pergunta 2: Qual a proporção de animais que chegam castrados? 3

3.3 Pergunta 3: Existe relação entre o canal de origem e a gravidade de saúde? 4

4 Modelagem do Banco de Dados Relacional (Backend) **4**

4.1 Regras de Negócio Implementadas 4

4.2 Modelo Entidade-Relacionamento e Esquema Relacional 5

4.3 Script DDL de Criação das Tabelas 5

4.4 Criação de Views 6

4.4.1 1. `vw_painel_animais` 6

4.4.2 2. `vw_adocoes_realizadas` 6

5 Manipulações nos Dados e Transações ACID **6**

5.1 Transação de Adoção (ACID) 7

6 Integração Backend (MySQL) e Frontend (Dashboard) **7**

7 Resultados Práticos: Imagens do Sistema e Funcionamento **8**

7.1 Consulta Geral e Monitoramento (READ via View) 8

7.2 Operação de Cadastro (INSERT) 8

7.3 Atualização de Estado (UPDATE) e Remoção (DELETE) 9

7.4 Adoção via Transação ACID 10

7.5 Análise Gráfica dos Dados Públicos 11

8 Considerações Finais **12**

1 Apresentação do Projeto

Este trabalho apresenta um sistema de banco de dados voltado para o controle de abrigo e adoção animal, estruturado em duas etapas práticas: a análise exploratória de dados reais de recolhimento de animais e o desenvolvimento de um sistema transacional para a rotina operacional de entrada, tratamento e adoção.

A rotina de um abrigo envolve desafios diários, como limitação de espaço físico, triagem médica rápida, controle de animais não castrados e o registro confiável das adoções. Sem um banco de dados estruturado, ocorrem falhas recorrentes, como animais registrados como disponíveis mesmo após já terem sido adotados, falta de histórico sobre a castração e ausência de prioridade para casos graves de saúde.

Para atender a essas necessidades, foram criados dois bancos de dados lógicos no MySQL 8.0:

1. **Banco Analítico (`db_analise_publica`):** Armazena os dados brutos de mais de 19 mil registros históricos do abrigo público de Austin, Texas, voltado para consultas e geração de relatórios.
2. **Banco Transacional (`db_abrigo_backend`):** Implementado na Terceira Forma Normal (3FN), atende às operações do dia a dia do abrigo (cadastro, atualização clínica, remoção e formalização de adoções via transações ACID).

O código-fonte completo, scripts SQL e arquivos de configuração estão no repositório:

`https://github.com/bendictoesterr/sistema-gestao-triagem-animal`

—

2 Descrição dos Dados Reais Utilizados e sua Origem

A base analítica utiliza os microdados abertos disponibilizados pelo portal de dados públicos de Austin, Texas (*Austin Animal Center Intakes*). O órgão é o maior centro público de acolhimento sob a diretriz *No-Kill* dos Estados Unidos, mantendo registros detalhados sobre as entradas de animais.

2.1 Especificações do Dataset

O extrato analisado contempla um histórico de 549 dias ininterruptos, totalizando 19.178 registros e 12 colunas na tabela `intakes_raw`:

- `animal_id` (VARCHAR): Identificador do animal no centro de acolhimento.
- `name` (VARCHAR): Nome do animal (nulo quando resgatado sem identificação).
- `datetime` (DATETIME): Data e hora da entrada.
- `found_location` (TEXT): Local ou via onde o animal foi recolhido.
- `intake_type` (VARCHAR): Canal de entrada (*Stray*, *Public Assist*, *Owner Surrender*, *Ambulance*, *Abandoned*).
- `intake_condition` (VARCHAR): Condição de saúde observada na triagem (*Normal*, *Injured*, *Sick*, *Nursing*, etc.).
- `animal_type` (VARCHAR): Tipo do animal (*Dog*, *Cat*, *Bird*, *Wildlife*, etc.).
- `sex_upon_intake` (VARCHAR): Sexo e condição reprodutiva (*Intact Female*, *Intact Male*, *Neutered Male*, *Spayed Female*, *Unknown*).

- `age_upon_intake` (VARCHAR): Idade aproximada no momento da entrada.
- `breed` (VARCHAR): Raça registrada.
- `color` (VARCHAR): Cor e padrão da pelagem.

2.2 Processo de Ingestão e Carga (ETL)

A importação dos dados brutos foi feita com o script Python `importar_csv.py`, utilizando o conector `mysql.connector`. Para garantir desempenho e integridade, as 19.178 linhas do arquivo CSV foram carregadas em blocos com `executemany`. Ao término, uma contagem simples confirmou que todas as 19.178 tuplas foram persistidas sem perdas.

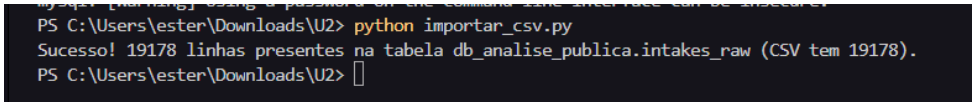


Figura 1: Confirmação da importação completa das 19.178 linhas no MySQL.

—

3 Análise dos Dados Reais: Contexto e Descobertas

A exploração da tabela analítica permitiu responder a três perguntas práticas que serviram para planejar as regras do sistema do abrigo:

3.1 Pergunta 1: Qual o volume e a variação diária de acolhimentos?

Para dimensionar a capacidade de acolhimento do abrigo, calculamos a média diária e os valores extremos de entrada:

```
SELECT
    COUNT(DISTINCT DATE(datetime)) AS total_dias,
    COUNT(*) AS total_entradas,
    ROUND(COUNT(*) / COUNT(DISTINCT DATE(datetime)), 2) AS media_diaria,
    MAX(entradas_dia) AS pico_diario,
    MIN(entradas_dia) AS menor_dia
FROM (
    SELECT DATE(datetime) AS dia, COUNT(*) AS entradas_dia
    FROM db_analise_publica.intakes_raw
    GROUP BY DATE(datetime)
) sub;
```

Interpretação: O abrigo teve média de **34,93 entradas por dia**. No entanto, a variação é alta: o menor dia teve apenas 1 entrada, enquanto o pico alcançou **96 entradas em um único dia**. Isso mostra que o abrigo precisa de um processo rápido de triagem para não sobrecarregar as instalações durante os picos.

3.2 Pergunta 2: Qual a proporção de animais que chegam castrados?

Analisamos a quantidade de animais que já entraram esterilizadoscirurgicamente (*Neutered* ou *Spayed*):

```
SELECT
    animal_type,
    COUNT(*) AS total_entradas,
    SUM(CASE WHEN sex_upon_intake LIKE 'Neutered%'
```

```
                OR sex_upon_intake LIKE 'Spayed%' THEN 1 ELSE 0 END) AS castrados,
    ROUND(100.0 * SUM(CASE WHEN sex_upon_intake LIKE 'Neutered%'
                           OR sex_upon_intake LIKE 'Spayed%' THEN 1 ELSE 0 END)
                      / COUNT(*), 2) AS pct_castrados
FROM db_analise_publica.intakes_raw
GROUP BY animal_type
ORDER BY total_entradas DESC;
```

Interpretação: Apenas **8,10%** dos gatos (total de 9.245) e **12,18%** dos cães (total de 6.837) entraram castrados. Em filhotes, o índice foi inferior a 0,5%. Esse resultado justificou a criação de um indicador visual de castração e uma rotina de controle pré-adoção no sistema do abrigo.

3.3 Pergunta 3: Existe relação entre o canal de origem e a gravidade de saúde?

Calculamos a taxa de animais com saúde crítica (*Injured*, *Sick*, *Medical Attention*) agrupados pelo canal de entrada:

```
SELECT
    intake_type,
    COUNT(*) AS total_casos,
    SUM(CASE WHEN intake_condition IN ('Injured', 'Sick', 'MED ATTN', 'MEDICAL')
             THEN 1 ELSE 0 END) AS casos_criticos,
    ROUND(100.0 * SUM(CASE WHEN intake_condition IN ('Injured', 'Sick', 'MED ATTN'
        , 'MEDICAL')
                           THEN 1 ELSE 0 END) / COUNT(*), 2) AS pct_criticos
FROM db_analise_publica.intakes_raw
GROUP BY intake_type
ORDER BY pct_criticos DESC;
```

Interpretação: O canal de resgate emergencial (*Ambulance*) registrou **29,65%** de animais em estado grave, enquanto animais recolhidos na rua (*Stray*) tiveram apenas **2,54%** de casos graves. Isso levou à criação da tabela de canais de entrada com classificação explícita de prioridade (*Baixa*, *Média*, *Alta*, *Crítica*).

—

4 Modelagem do Banco de Dados Relacional (Backend)

Com base nos resultados analíticos, foi modelado o banco de dados transacional `db_abrigo_backend` para atender à rotina diária de cadastro, atendimento veterinário e adoção.

4.1 Regras de Negócio Implementadas

RN1. Dados Obrigatórios: Todo animal deve ter nome, espécie, raça, sexo e canal de entrada informado.

RN2. Status Controlado: O status de adoção do animal assume apenas: *Disponível*, *Em Tratamento* ou *Adotado*.

RN3. Restrição de Adoção: Animais marcados como *Em Tratamento* não podem ser adotados até sua liberação clínica.

RN4. Operação Atômica de Adoção: A adoção registra o tutor e altera o status do animal para *Adotado* na mesma transação. Se uma etapa falhar, nenhuma alteração é salva.

RN5. Integridade Referencial: Não é permitido remover um canal de entrada enquanto houver animais associados a ele (`ON DELETE RESTRICT`).

4.2 Modelo Entidade-Relacionamento e Esquema Relacional

O modelo conta com três tabelas normalizadas em 3FN:

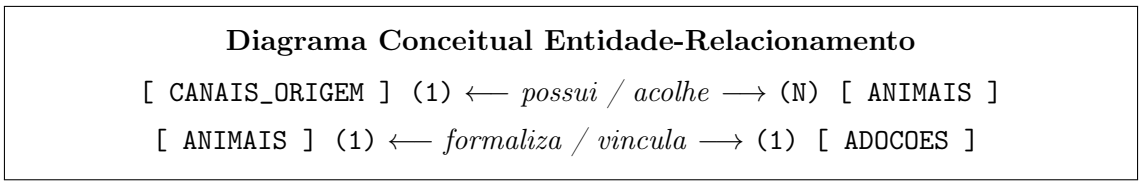


Figura 2: Diagrama de relacionamentos do banco transacional.

- `canais_origem` (id_canal, `nome_canal`, `descricao`, `nivel_prioridade`): Registra as portas de entrada e o grau de prioridade de atendimento.
- `animais` (id_animal, `nome`, `especie`, `raca`, `sexo`, `castrado`, `condicao_saude`, `status_adocao`, `data_entrada`, *id_canal*): Tabela dos animais acolhidos, com chave estrangeira para `canais_origem`.
- `adocoes` (id_adocao, *id_animal*, `nome_adotante`, `telefone`, `data_adocao`): Registra a saída do animal para o tutor, com restrição `UNIQUE` no `id_animal` para impedir que o mesmo animal receba adoções duplicadas.

4.3 Script DDL de Criação das Tabelas

```
CREATE DATABASE IF NOT EXISTS db_abrigo_backend
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

USE db_abrigo_backend;

CREATE TABLE IF NOT EXISTS canais_origem (
    id_canal INT AUTO_INCREMENT PRIMARY KEY,
    nome_canal VARCHAR(80) NOT NULL UNIQUE,
    descricao VARCHAR(255) NULL,
    nivel_prioridade ENUM('Baixa', 'Media', 'Alta', 'Critica') NOT NULL DEFAULT '
        Media'
) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS animais (
    id_animal INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(60) NOT NULL,
    especie ENUM('Dog', 'Cat', 'Puppy', 'Kitten', 'Other') NOT NULL,
    raca VARCHAR(100) NOT NULL DEFAULT 'Mixed / SRD',
    sexo ENUM('Male', 'Female', 'Unknown') NOT NULL,
    castrado TINYINT(1) NOT NULL DEFAULT 0,
    condicao_saude ENUM('Normal', 'Injured', 'Sick', 'Medical Attention') NOT NULL
        DEFAULT 'Normal',
    status_adocao ENUM('Disponivel', 'Em Tratamento', 'Adotado') NOT NULL DEFAULT
        'Disponivel',
    data_entrada DATE NOT NULL,
    id_canal INT NOT NULL,
    CONSTRAINT fk_animais_canal FOREIGN KEY (id_canal)
        REFERENCES canais_origem (id_canal)
        ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS adocoes (
    id_adocao INT AUTO_INCREMENT PRIMARY KEY,
    id_animal INT NOT NULL UNIQUE,
    nome_adotante VARCHAR(100) NOT NULL,
    telefone VARCHAR(30) NULL,
    data_adocao DATE NOT NULL,
    CONSTRAINT fk_adocoes_animal FOREIGN KEY (id_animal)
```

```
        REFERENCES animais (id_animal)
        ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB;
```

4.4 Criação de Views

Foram criadas duas views para simplificar as consultas da interface e evitar repetição de código SQL no frontend:

4.4.1 1. vw_painel_animais

Reúne os dados do animal com o nome do canal e nível de prioridade, formatando o booleano de castração para exibição:

```
CREATE OR REPLACE VIEW vw_painel_animais AS
SELECT
    a.id_animal,
    a.nome,
    a.especie,
    a.raca,
    a.sexo,
    CASE WHEN a.castrado = 1 THEN 'Sim' ELSE 'Nao' END AS castrado_fmt,
    a.condicao_saude,
    a.status_adocao,
    a.data_entrada,
    c.nome_canal AS canal_origem,
    c.nivel_prioridade
FROM animais a
JOIN canais_origem c ON a.id_canal = c.id_canal
ORDER BY a.data_entrada DESC, a.id_animal DESC;
```

4.4.2 2. vw_adocoes_realizadas

Combina dados da adoção e do animal para exibir o histórico recente com o contato do adotante:

```
CREATE OR REPLACE VIEW vw_adocoes_realizadas AS
SELECT
    ad.id_adocao,
    ad.id_animal,
    an.nome AS animal,
    an.especie,
    ad.nome_adotante,
    ad.telefone AS telefone_adotante,
    ad.data_adocao
FROM adocoes ad
JOIN animais an ON ad.id_animal = an.id_animal
ORDER BY ad.data_adocao DESC;
```

—

5 Manipulações nos Dados e Transações ACID

O sistema cobre todas as operações básicas de banco de dados (CRUD) e usa controle transacional do motor InnoDB:

- **Create (INSERT):** Inserção de novos animais com validação de campos e chave gerada automaticamente.

- **Read (SELECT via Views):** Consulta dos animais filtrados por status diretamente pelas views.
- **Update (UPDATE):** Atualização de status e condição de saúde por meio do identificador `id_animal`.
- **Delete (DELETE):** Exclusão com confirmação explícita na tela para evitar perda acidental de registros.

5.1 Transação de Adoção (ACID)

A formalização da adoção exige salvar o adotante e atualizar o animal como adotado. As duas ações são executadas em uma única transação atômica:

```
conn = get_db_connection("db_abrigo_backend")
cursor = conn.cursor()
try:
    conn.start_transaction()

    # 1. Registra a adocao
    sql_adocao = """
        INSERT INTO adocoes (id_animal, nome_adotante, telefone, data_adocao)
        VALUES (%s, %s, %s, %s)
    """
    cursor.execute(sql_adocao, (id_animal, nome_adotante, telefone, data_adocao))

    # 2. Atualiza o status do animal
    sql_update = """
        UPDATE animais
        SET status_adocao = 'Adotado'
        WHERE id_animal = %s
    """
    cursor.execute(sql_update, (id_animal,))

    conn.commit() # Confirma ambas as alteracoes
except Exception as e:
    conn.rollback() # Desfaz tudo se houver falha
    raise e
finally:
    cursor.close()
    conn.close()
```

Isso assegura que não existam registros órfãos de adoção e que nenhum animal permaneça com status incorreto.

—

6 Integração Backend (MySQL) e Frontend (Dashboard)

A arquitetura separa a interface da camada de persistência:

- **Infraestrutura via Docker:** O MySQL 8.0 roda em um contêiner configurado com persistência de volume (`mysql_abrigo_data`) e conjunto de caracteres `utf8mb4`, garantindo integridade e suporte à acentuação.
- **Aplicação Python e Streamlit:** A interface conecta-se ao banco pelo driver oficial `mysql-connector-python`. Consultas analíticas utilizam cache temporário (`@st.cache_data`) para carregamento rápido dos dados sem sobrecarregar o banco.

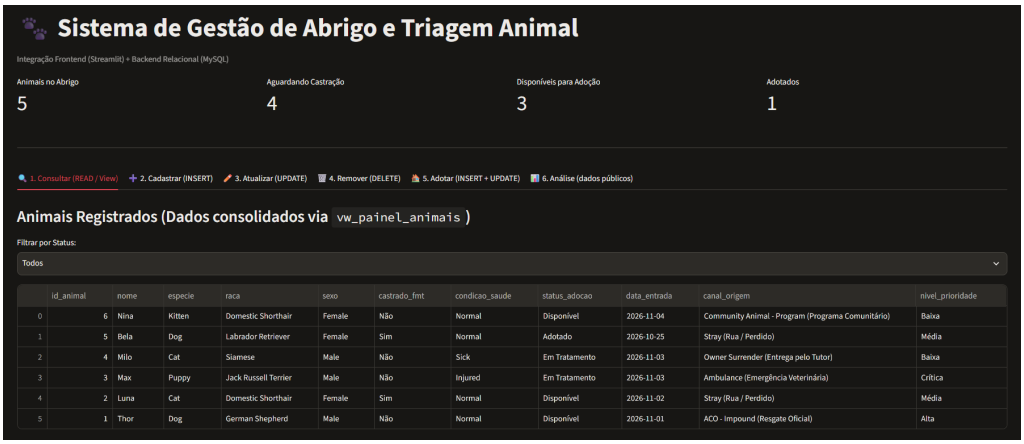
—

7 Resultados Práticos: Imagens do Sistema e Funcionamento

Abaixo são apresentadas as telas do painel demonstrando o fluxo operacional e as análises:

7.1 Consulta Geral e Monitoramento (READ via View)

A tela inicial consome a view `vw_painel_animais` e exibe os cartões com o resumo dos contadores no topo do painel.

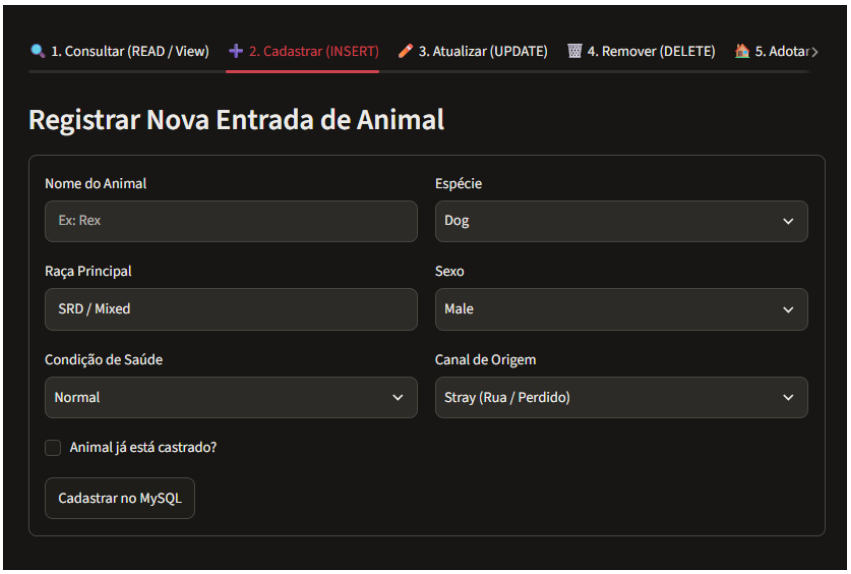


	id_animal	nome	especie	raca	sexo	castrado_fmt	condicao_saude	status_adocao	data_entrada	canal_origem	nivel_prioridade
0	6	Nina	Kitten	Domestic Shorthair	Female	Não	Normal	Disponível	2026-11-04	Community Animal - Program (Programa Comunitário)	Baixa
1	5	Bela	Dog	Labrador Retriever	Female	Sim	Normal	Adotado	2026-10-25	Stray (Rua / Perdido)	Média
2	4	Milo	Cat	Siamese	Male	Não	Sick	Em Tratamento	2026-11-03	Owner Surrender (Entrega pelo Tutor)	Baixa
3	3	Max	Puppy	Jack Russell Terrier	Male	Não	Injured	Em Tratamento	2026-11-03	Ambulance (Emergência Veterinária)	Crítica
4	2	Luna	Cat	Domestic Shorthair	Female	Sim	Normal	Disponível	2026-11-02	Stray (Rua / Perdido)	Média
5	1	Thor	Dog	German Shepherd	Male	Não	Normal	Disponível	2026-11-01	ACO - Impound (Resgate Oficial)	Alta

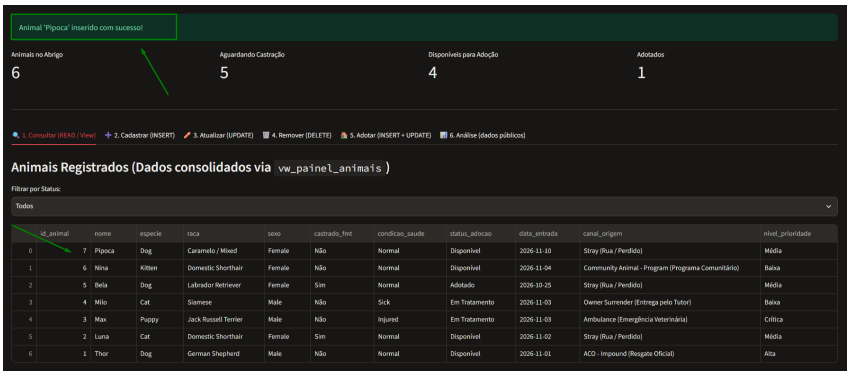
Figura 3: Tela inicial de monitoramento dos animais cadastrados.

7.2 Operação de Cadastro (INSERT)

O formulário recolhe os dados da admissão e executa a inserção. As Figuras 4a e 4b mostram a entrada dos dados e o registro concluído no banco.



(a) Formulário de cadastro.



(b) Confirmação da inserção e atualização das métricas.

Figura 4: Fluxo de cadastramento de animal no banco de dados.

7.3 Atualização de Estado (UPDATE) e Remoção (DELETE)

A Figura 5 apresenta a atualização de status do animal Thor (ID 1) para *"Em Tratamento"*. A Figura 6 exibe a exclusão confirmada do cadastro de Pipoca (ID 7) após confirmação.

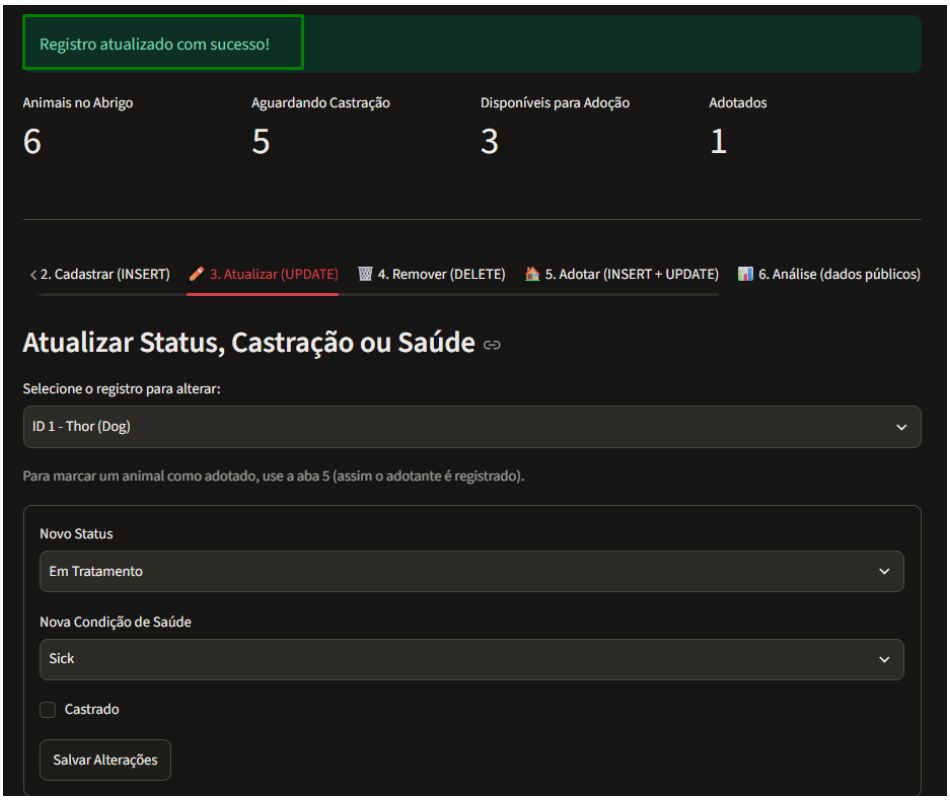


Figura 5: Atualização de status e condição de saúde no banco.

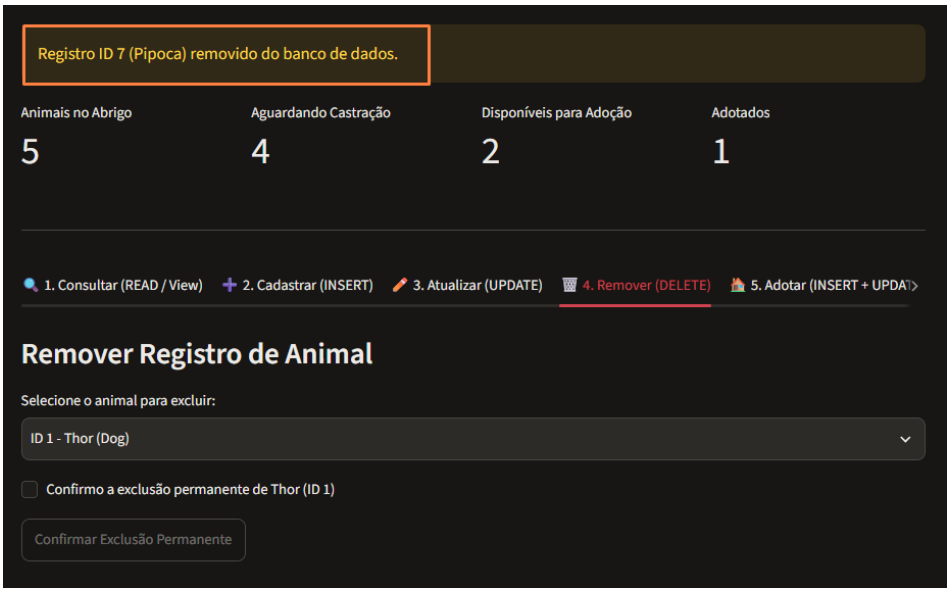


Figura 6: Remoção com confirmação de segurança.

7.4 Adoção via Transação ACID

A Figura 7 demonstra a confirmação da adoção da felina Luna (ID 2), salvando o adotante e alterando o status do animal de forma atômica.

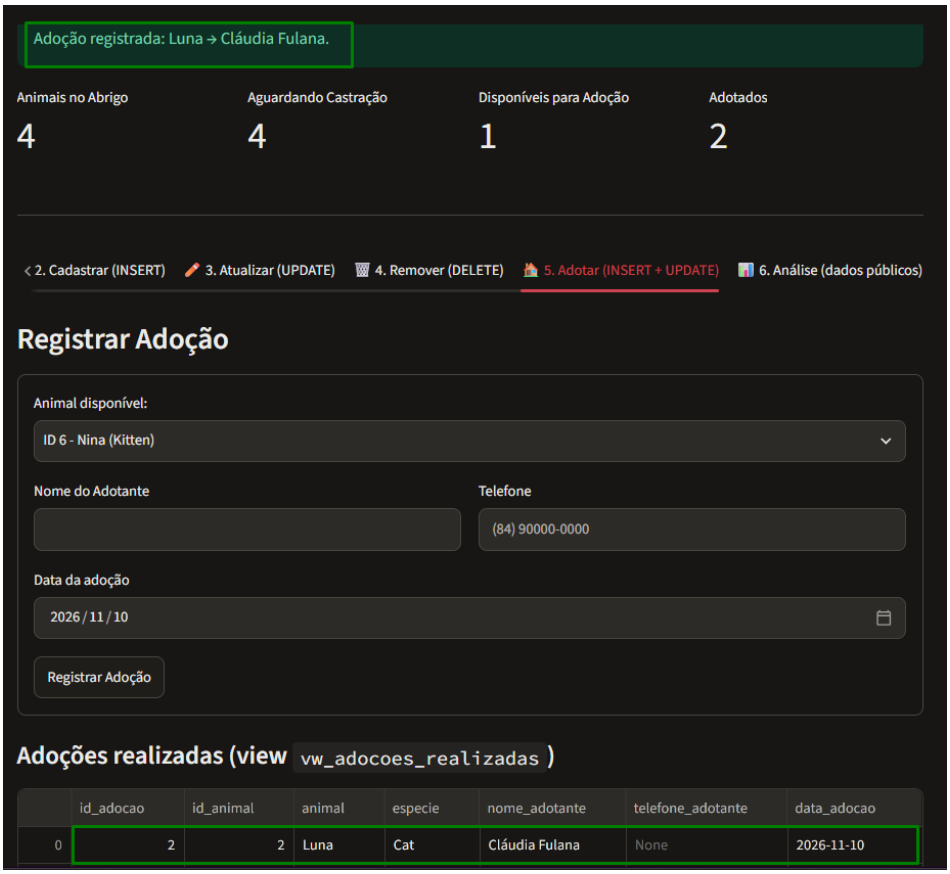


Figura 7: Registro de adoção executado em bloco transacional.

7.5 Análise Gráfica dos Dados Públicos

As Figuras 8 e 9 exibem as métricas gerais do conjunto público de dados, a distribuição por espécies e a taxa de casos críticos por canal de resgate.

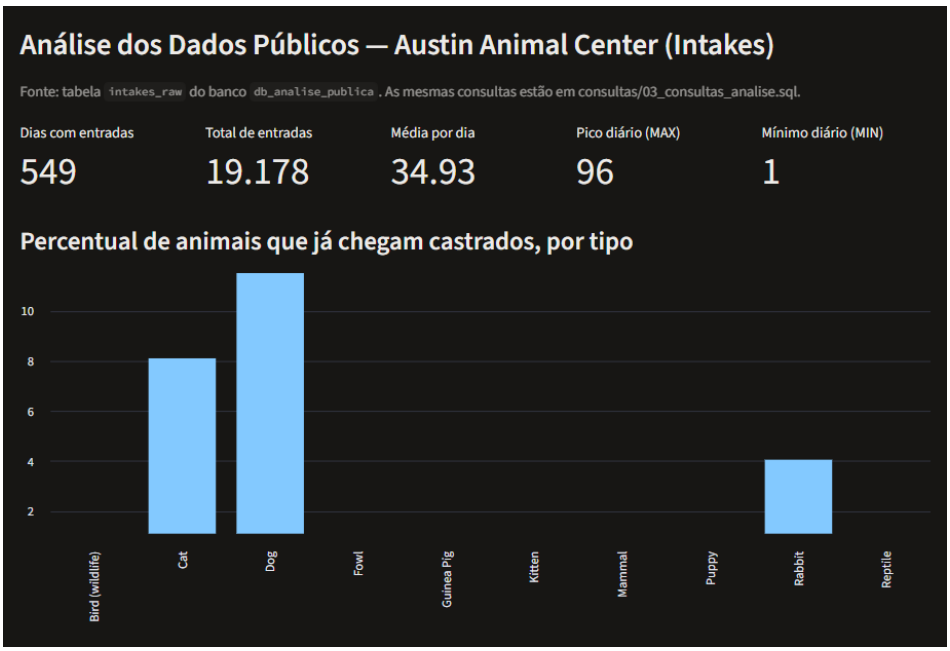
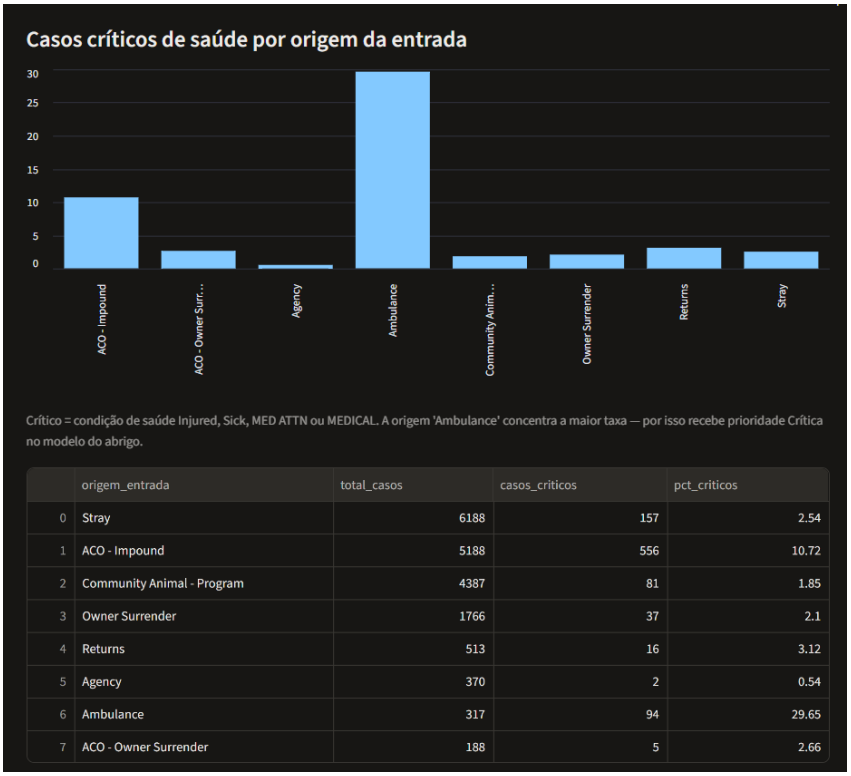


Figura 8: Indicadores gerais e taxa de castração ao ingresso.

	tipo_animal	total_entradas	castrados	nao_castrados	percentual_castrados
0	Cat	9245	749	8496	8.1
1	Dog	6837	833	6004	12.18
2	Kitten	1696	6	1690	0.35
3	Mammal	741	0	741	0
4	Puppy	337	0	337	0
5	Rabbit	74	3	71	4.05
6	Guinea Pig	51	0	51	0
7	Reptile	38	0	38	0
8	Fowl	38	0	38	0
9	Bird (wildlife)	32	0	32	0

(a) Distribuição por espécies e proporção de castração.



	origem_entrada	total_casos	casos_criticos	pct_criticos
0	Stray	6188	157	2.54
1	ACO - Impound	5188	556	10.72
2	Community Animal - Program	4387	81	1.85
3	Owner Surrender	1766	37	2.1
4	Returns	513	16	3.12
5	Agency	370	2	0.54
6	Ambulance	317	94	29.65
7	ACO - Owner Surrender	188	5	2.66

(b) Casos graves de saúde por canal de entrada.

Figura 9: Análise das entradas históricas do Austin Animal Center.

8 Considerações Finais

O trabalho cobriu o ciclo de desenvolvimento de banco de dados, partindo do estudo de uma base pública com 19.178 tuplas até a entrega de um sistema relacional funcional.

As informações obtidas na análise exploratória (como a baixa taxa de castração prévia e a maior incidência de problemas graves em resgates emergenciais) ajudaram a definir as tabelas, tipos de dados e restrições do banco transacional. A arquitetura utilizando MySQL 8.0 em Docker e interface web em Python com Streamlit permitiu testar as operações do dia a dia com integridade de dados e segurança transacional.